

Implementation and Comparative Analysis of K-Means Clustering: Sequential and Parallel Version

Alessio Poggesi

alessio.poggesi@edu.unifi.it

Abstract

K-Means clustering is a widely used unsupervised learning algorithm, but its iterative nature makes it computationally demanding on large datasets. In this work we implement and benchmark a sequential C++ K-Means baseline and two OpenMP parallel variants that differ in memory layout: an Array of Structures (AoS) implementation and a Structure of Arrays (SoA) implementation. Both parallel versions exploit OpenMP `parallel` for regions with `schedule(runtime)` and thread-local accumulators for centroid updates, enabling a systematic exploration of runtime scheduling policies and chunk sizes. Experiments are performed on multiple synthetic datasets (from 10^4 up to 10^6 points) and different numbers of clusters, collecting repeated measurements (mean, standard deviation, min/max) after a warm-up phase and using deterministic centroid initialization to ensure fair comparisons across runs. Correctness is validated by comparing the final sum of squared errors (SSE) against the sequential reference within a tight relative tolerance. Overall, the SoA variant provides the best performance and reaches a maximum speedup of about $4.60\times$ over the sequential baseline on the largest dataset (10^6 points, $k = 20$, 8 threads), highlighting the impact of data layout and memory access patterns on OpenMP scalability.

Future Distribution Permission

The author of this report gives permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

K-Means is one of the most widely adopted clustering algorithms in unsupervised learning, used in domains such as exploratory data analysis, image segmentation, and data summarization. Given a set of N points, K-Means iteratively assigns each point to the closest of K centroids and

then updates each centroid as the mean of its assigned points. Despite its conceptual simplicity, K-Means can become computationally expensive when either the dataset size or the number of clusters grows, making performance a key concern in practical workloads.

From a computational perspective, the dominant cost is typically the assignment step: for each iteration, distances must be computed between every point and every centroid, leading to a per-iteration complexity on the order of $\mathcal{O}(N \cdot K \cdot D)$ (with D the number of dimensions). This step is highly parallelizable, but achieving scalable performance on modern multicore CPUs is not trivial. Practical implementations must contend with memory bandwidth limitations, cache behavior, and synchronization overheads during centroid updates (which require reductions over all points). Moreover, parallel performance can be strongly affected by runtime choices such as OpenMP scheduling policy and chunk size. In this project, we explicitly explore these parameters using `schedule(runtime)` and varying `omp_set_schedule` configurations.

The objective of this work is to design, implement, and experimentally evaluate a K-Means solution in C++ that highlights how algorithmic structure and data layout influence OpenMP scalability. We provide a sequential baseline and two shared-memory OpenMP variants that differ in memory organization: an *Array of Structures* (AoS) and a *Structure of Arrays* (SoA) approach. Both variants parallelize the assignment phase and perform centroid updates via thread-local accumulation followed by a merge step, en-

abling a controlled comparison focused on memory access patterns and parallel overheads.

In terms of scope, the study targets shared-memory execution on a single multicore machine (OpenMP), and focuses on: (i) measuring speedup and scalability over multiple input sizes and cluster counts, (ii) comparing AoS vs SoA as two parallelization/data-layout strategies, and (iii) analyzing how scheduling and chunk size impact performance and where scalability saturates. Correctness is validated by comparing the final Sum of Squared Errors (SSE) against the sequential reference under a tight tolerance.

2. Algorithm Description

2.1. Sequential K-Means

Given a dataset of N points and a target number of clusters K , the sequential implementation iterates two phases until convergence:

1. **Assignment step.** For each point p_i , compute its squared Euclidean distance to every centroid c_j and assign the label of the closest centroid:

$$\ell_i = \arg \min_{j \in \{0, \dots, K-1\}} \|p_i - c_j\|^2.$$

In our code, this is implemented as a nested loop over points and centroids, storing the result in a `labels` vector.

2. **Update step.** Recompute each centroid as the mean of the points assigned to it. This is done by accumulating, for each cluster j , the sum of coordinates of assigned points and the corresponding count, and then dividing sums by counts.

The outer loop stops either when a maximum number of iterations is reached (`max_iters = 600`) or when the algorithm converges according to a geometric criterion: the maximum centroid displacement between two consecutive iterations is below a threshold ($\varepsilon = 10^{-6}$, checked as ε^2). The sequential implementation, together with the two OpenMP variants, follows a common interface exposing `assign_clusters()`, `update_centroids()`, and `fit()`.

2.2. Parallelizable sections

K-Means is naturally amenable to shared-memory parallelism because its dominant operations are data-parallel:

- **Parallel assignment.** The assignment step is independent across points: each point can be processed in parallel, since labels are written to distinct indices. Both OpenMP variants parallelize this phase with a `parallel for` over the point index and `schedule(runtime)`.
- **Parallel centroid accumulation.** The update step contains a large loop over points that accumulates per-cluster sums and counts. A naive parallelization would introduce contention (multiple threads updating the same cluster). To avoid this, our OpenMP implementations use *thread-local* accumulators (arrays of sums and counts per thread) followed by a merge step that combines partial results.
- **Parallel centroid finalization.** Once global sums and counts are available, updating the K centroids (division by counts and write-back) is also parallelizable as a loop over clusters.

These observations motivate our two parallel variants (AoS and SoA): both exploit the same algorithmic parallel structure, while differing in data layout and memory access patterns, allowing us to study their impact on scalability and performance.

3. Parallel Implementation

3.1. Parallelization strategy

Both OpenMP variants follow the same high-level strategy: parallelize the *assignment* phase over points and parallelize the *centroid update* using thread-private accumulators to avoid contention. The two implementations differ mainly in memory layout (AoS vs SoA), while keeping the algorithmic structure comparable.

3.2. OpenMP directives used

Point assignment (`assign_clusters`). In both variants, cluster assignment is implemented as a `#pragma omp parallel for` over the point index, with `schedule(runtime)` so that the scheduling policy (static/dynamic) and chunk size can be controlled externally (from `main.cpp` via `omp_set_schedule`).

Centroid update (`update_centroids`). The update phase is split into: (i) a parallel loop over points that accumulates partial sums and counts in thread-local arrays, and (ii) a merge + finalization step. The accumulation uses `#pragma omp parallel plus` `#pragma omp for` `schedule(runtime)`.

Centroid finalization. Once global sums and counts are available, updating the K centroids is implemented as a `#pragma omp parallel for` `schedule(runtime)` over the cluster index.

No OpenMP reductions. A direct reduction is not used for centroid updates because the target is a set of per-cluster accumulators indexed by labels (indirect updates). Instead, thread-local buffers are employed and then merged.

3.3. Synchronization mechanisms

The main synchronization points are:

- **Implicit barriers** at the end of OpenMP `for` regions (ensuring all labels / partial sums are ready before proceeding).
- **A sequential merge step** (performed outside the parallel region) that combines the thread-local accumulators into global sums and counts. This avoids fine-grained synchronization (atomics/critical) inside the hot loop.

In the AoS version we also explicitly determine the actual number of threads used via a small `parallel` region and a `single` section (so that local buffers match the runtime thread count). In

the SoA version, local buffers are allocated using `omp_get_max_threads()`, which is safe but may over-allocate if fewer threads are used at runtime.

3.4. Vectorization techniques applied

The SoA implementation is designed to be more SIMD-friendly because point coordinates are stored in separate contiguous arrays (`x[]` and `y[]`), reducing gather-like access patterns. While the code contains a note about using `#pragma omp simd` in the centroid loop, it is intentionally not enabled because a correct SIMD min-reduction over both distance and best-cluster index is non-trivial and can easily become incorrect. Therefore, SIMD relies on compiler auto-vectorization where applicable, and on improved memory locality from the SoA layout.

3.5. Data structures and memory layout

AoS. Points are stored as a vector of structs (`std::vector<Point>`), where `Point` contains `x`, `y`. This is simple and natural, but may lead to less efficient streaming when only a subset of fields is used.

SoA. Points are stored as `PointsSoA` with two arrays `x[]` and `y[]`. Centroids are also mirrored into `centroid_x[]` and `centroid_y[]` for faster access during assignment. After each update, the SoA centroids are copied back into the AoS centroid vector to reuse common code for printing and convergence checks.

Dataset loading. The dataset loader populates *both* representations (`points` and `points_soa`) from the same CSV input, enabling fair comparisons between AoS and SoA without changing the input pipeline.

3.6. Challenges encountered and solutions

- **Contention in centroid updates.** Multiple points can map to the same cluster, so naive shared updates would require atomics or critical sections. We avoid this by using thread-

local sums/counts and a merge step, which keeps the hot loop lock-free.

- **Empty clusters.** When a cluster receives no points, the AoS implementation keeps the previous centroid (common practical choice). The SoA version updates only when `count[j] > 0`, effectively preserving the previous values as well.
- **Reproducible benchmarking.** Scheduling is controlled via `schedule(runtime)` and configured from the benchmark driver using `omp_set_schedule(kind, chunk)`, so we can systematically study the effect of policy and chunk size.

4. Experimental Setup

4.1. Hardware Specifications

All experiments were executed on a single shared-memory machine (device name: `katana`) running a 64-bit Windows operating system on an x86_64 processor. The system is equipped with an 11th Gen Intel® Core™ i7-11800H CPU (base frequency 2.30 GHz) and 16 GB of installed RAM.

In the benchmark campaign, we explored up to 32 OpenMP threads (`THREAD_LIST={1, 2, 4, 8, 16, 32}`). This allows evaluating both scalability within the available CPU parallelism and the effects of oversubscription at high thread counts.

4.2. Software Environment

The project is built with CMake and targets the C++17 standard. OpenMP support is required and the OpenMP runtime is linked to both the benchmark executable and the correctness executable. Release builds enable compiler optimizations and OpenMP (e.g., `-O3`, `-march=native`, `-ftree-vectorize`, `-fopenmp` for GCC/Clang toolchains).

Execution times are measured using `omp_get_wtime()` and converted to milliseconds, providing a low-overhead wall-clock timer consistent with the OpenMP runtime.

4.3. Datasets

We use synthetic 2D datasets stored as CSV files containing one point per row with two columns (`x, y`). The loader accepts either a header row or raw numeric data, and populates both an Array-of-Structures (AoS) representation (vector of `Point{x, y}`) and a Structure-of-Arrays (SoA) representation (two vectors `x[]` and `y[]`).

Datasets were generated through a dedicated Python script (`generate_blobs`), using the `make_blobs` generator from `scikit-learn` to create 2D Gaussian clusters. In our configuration, blob centers are sampled uniformly within a fixed box (`center_box = [-100, 100]` in both dimensions), and all clusters share the same standard deviation (`cluster_std = 10.0`). A deterministic seed is used for each (N, K) pair to ensure reproducibility across runs.

For performance tests, datasets are selected through a fixed hard-coded list of filenames (6 input sizes: $N = 10k, 50k, 100k, 300k, 500k, 1M$; and 3 cluster counts: $K = 6, 10, 20$), for a total of 18 datasets. Any additional generated datasets are ignored unless explicitly included in this list.

Each benchmark run loads the dataset from disk and initializes the centroids before timing. The reported runtime measures only the K-Means compute phase (the `fit()` call), excluding CSV I/O, parsing, and centroid initialization overhead.

4.4. Parameters Tested

For each dataset, the benchmark explores the following parameters:

- **Variant / memory layout:** AoS vs SoA.
- **Thread count:** `THREAD_LIST={1, 2, 4, 8, 16, 32}`.
- **Scheduling policy:** `static` and `dynamic`.
- **Chunk size:** `CHUNKS={0, 1, 32, 128}`, where 0 selects the OpenMP default.

Scheduling is applied in the driver via `omp_set_schedule(kind, chunk)`, while OpenMP kernels use `schedule(runtime)` so

that the runtime schedule is consistently enforced across all parallel regions.

Algorithm-side stopping criteria are kept identical across sequential and parallel variants: a maximum of `max_iters=600` iterations and a convergence threshold `eps=1e-6` (evaluated via squared centroid shift).

4.5. Measurement Methodology

For each configuration (dataset, layout, threads, scheduling policy, chunk size), the benchmark performs `NUM_RUNS=5` executions and discards the first `WARMUP_RUNS=1` as warm-up to reduce transient effects.

To reduce noise due to different initial centroid placements, each run uses a deterministic seed (`seed = 12345 + run`), and the same initialization is applied to both sequential and parallel executions within the same repetition.

For every configuration, we collect runtime statistics (mean, standard deviation, minimum, maximum) for both sequential and parallel variants and additionally record the number of iterations to convergence to ensure comparisons are not biased by different convergence behavior. All results are written to a CSV file (`resultati_openmp_aos_vs_soa.csv`) with a schema that includes both timing and iteration statistics.

Finally, a separate correctness executable is provided to validate numerical consistency across the three implementations (Sequential, OpenMP AoS, OpenMP SoA). It compares the sum of squared errors (SSE) against the sequential reference using a relative tolerance (e.g., `rel_eps=1e-9`) and reports per-run pass/fail outcomes.

5. Results and Analysis

5.1. Correctness Validation

Correctness was verified with a dedicated executable that runs the three implementations (Sequential, OpenMP–AoS, OpenMP–SoA) on the same reference dataset (`N500000_K10.csv`), using the same centroid initialization and a fixed

OpenMP schedule (`static`, default chunk size). For each measured run, we report (i) the number of iterations to convergence and (ii) the SSE. The OK/FAIL outcome is determined by comparing the SSE of each parallel variant against the sequential reference under a relative tolerance; iteration counts are reported to highlight potential convergence differences.

5.2. Performance Results

All performance results are computed from the benchmark harness by taking the mean runtime over multiple repetitions (after a warm-up) for each configuration. We define the speedup as:

$$S(T) = \frac{\bar{t}_{\text{SEQ}}}{\bar{t}_{\text{OMP}}(T)},$$

where T is the number of OpenMP threads, and $\bar{t}$ denotes the average runtime.

5.2.1 Speedup Curves (Speedup vs Threads, Multiple Input Sizes)

The speedup curves show the expected behavior for a shared-memory implementation of K-Means:

- **Small inputs do not scale.** For $N = 10\text{k}$, overheads (parallel region setup, scheduling, and reductions/merges) dominate, and speedup is close to $1\times$ (or even below $1\times$ at higher thread counts).
- **Scaling improves with larger N .** For medium and large datasets, speedup increases rapidly up to a moderate thread count, reaching the best results on the largest inputs.
- **Diminishing returns at high T .** Beyond a certain point, the curves saturate (and in some cases slightly degrade), indicating that the implementation becomes limited by memory bandwidth and synchronization/merge overheads rather than pure compute.

As a representative example (largest input, $N = 10^6$, $K = 20$), the best observed speedups are approximately:

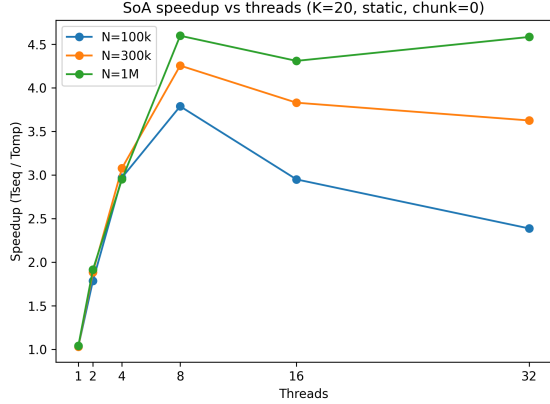


Figure 1. SoA speedup vs threads for multiple input sizes ($K = 20$, `static`, `chunk=0`).

Table 1. Best speedup for the SoA variant at $K = 20$ (best configuration across scheduling/chunk for each N).

N	Best T	Schedule	Chunk	Best speedup
10k	1	static	128	$\approx 1.00\times$
50k	8	static	0	$\approx 2.90\times$
100k	8	static	0	$\approx 3.79\times$
300k	8	static	0	$\approx 4.26\times$
500k	8	static	128	$\approx 4.31\times$
1M	8	static	0	$\approx 4.60\times$

- **OpenMP–SoA:** up to $\approx 4.60\times$ (peak around 8 threads),
- **OpenMP–AoS:** up to $\approx 4.34\times$ (peak at higher thread counts, but still below SoA in the best configuration).

5.2.2 Scalability Analysis (Strong and Weak Scaling)

Strong scaling. We evaluate strong scaling by fixing (N, K) and varying T . On the largest instance ($N = 10^6$, $K = 20$), SoA scales well up to 8 threads (peak $\approx 4.60\times$), after which improvements are marginal and efficiency drops:

$$E(T) = \frac{S(T)}{T} \Rightarrow$$

$$E(8) \approx 0.57, E(16) \approx 0.27, E(32) \approx 0.14.$$

This saturation indicates that the parallel fraction is high, but scalability is constrained by shared resources (primarily memory bandwidth) and by the cost of combining partial results in the centroid update.

Weak scaling (approximate). A textbook weak-scaling study requires increasing N proportionally to T while keeping the work per thread constant. Since our dataset grid is defined by a fixed list of (N, K) pairs, we cannot form a perfect weak-scaling series. However, the trend across increasing N (Table 1) still provides an important practical insight: **larger problems amortize fixed parallel overheads better**, yielding higher effective speedups, especially for the SoA variant.

5.2.3 Effect of OpenMP Parameters (Scheduling and Chunk Size)

The scheduling policy and chunk size have a major impact on performance:

- **Static scheduling is consistently best.** The workload per point is fairly uniform in both phases (assignment and accumulation), therefore static scheduling achieves good load balance with minimal overhead.
- **Dynamic scheduling can be catastrophic with the default chunk.** With `dynamic` and the OpenMP default chunk (often 1), the runtime spends a large fraction of time in scheduling and synchronization. On large inputs this can completely destroy performance, yielding speedups well below $1\times$.
- **Large chunks mitigate dynamic overhead, but remain inferior to static.** Using `dynamic` with a larger chunk (e.g., 128) significantly reduces scheduling overhead, yet static still dominates for our kernels.
- **Chunk size matters also for static.** Very small chunks (e.g., 1) increase overhead and reduce locality. In contrast, the default static chunk (0) is typically the best or near-best choice.

Overall, the best-performing configurations are overwhelmingly `static` with `chunk=0`, while `dynamic` requires a large chunk to be competitive at all.

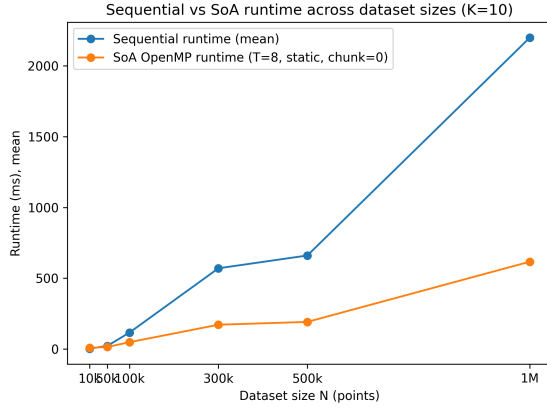


Figure 2. Sequential vs SoA runtime across dataset sizes ($K = 10$). SoA configuration: $T = 8$, *static*, *chunk=0*.

5.2.4 Comparison of Variants: AoS vs SoA

Across the benchmark space, the SoA implementation is generally faster than AoS, and its advantage is most pronounced when K is small (lower arithmetic intensity, more memory-bound):

- **SoA improves memory locality and bandwidth utilization** in the assignment step by streaming two contiguous arrays (x and y), which is more cache-friendly than loading interleaved structures.
- **The relative benefit shrinks as K grows.** For larger K , each point performs more arithmetic per memory load (distance evaluations against more centroids), increasing compute intensity and reducing the impact of memory layout.

On the largest dataset ($N = 10^6$), SoA achieves the overall best speedup and is the most robust choice across thread counts.

5.2.5 Bottleneck Analysis: What Limits Scalability?

The main scalability limiters observed in the results are:

- **Memory bandwidth saturation.** K-Means performs multiple streaming passes over the dataset per iteration. After a moderate number of threads, additional threads mostly compete for the same memory bandwidth, yielding diminishing returns.

- **Centroid update merge overhead.** The update phase uses thread-local accumulators followed by a merge step. While the merge is not the dominant cost at low T , it becomes more visible as the assignment step accelerates.
- **Parallel runtime overheads (especially scheduling).** Dynamic scheduling with small chunks introduces significant overhead and lock contention, severely limiting performance.
- **Oversubscription effects at high T .** Using a thread count that approaches or exceeds the machine’s effective hardware parallelism can increase context switching and cache contention, which explains why speedup may plateau or slightly degrade beyond the best T .

Key takeaway. The best overall configuration in our study is the **OpenMP-SoA** variant with **static scheduling** and an appropriate chunk choice (typically *chunk=0*), reaching a peak speedup of about $4.6\times$ on the largest input.

6. Discussion and Conclusions

6.1. Discussion

The experimental results confirm the expected strong-scaling behavior of a shared-memory OpenMP implementation of K-Means. Speedup improves rapidly when increasing the number of threads up to a moderate level (typically around 8 threads on our platform), and then tends to saturate or fluctuate at higher thread counts. This trend is consistent with the theoretical expectations of Amdahl’s Law: even if the assignment step is highly parallel, the overall speedup is bounded by non-parallelizable components and by parallel overheads (barriers, scheduling, and the centroid-update merge). In addition, the plateau at high thread counts suggests that the computation becomes increasingly memory-bound, meaning that adding threads mainly increases contention on the shared memory hierarchy rather than providing proportional gains.

The observed scaling also aligns with Gustafson’s perspective: larger problem sizes obtain better effective speedups because fixed overheads are amortized and the parallel fraction becomes dominant. This is clearly visible in the speedup curves across multiple dataset sizes, where the largest input ($N = 10^6$) achieves the highest speedup, while small datasets (e.g., $N = 10k$) do not benefit significantly from parallelization. From the OpenMP parameter exploration, `static` scheduling consistently provides the best performance, as the workload per point is fairly uniform; in contrast, `dynamic` scheduling with small chunks introduces high runtime overhead and can severely reduce speedup.

Regarding implementation choices, the Structure-of-Arrays (SoA) layout is generally the most robust and fastest variant. The advantage of SoA is coherent with hardware considerations: storing coordinates in contiguous arrays improves spatial locality, reduces cache misses, and enables more efficient memory streaming during the distance-computation phase. However, the relative gap between SoA and AoS tends to shrink at higher concurrency due to memory bandwidth saturation and the increasing weight of synchronization/merge costs in the update step. Finally, thread counts beyond the machine’s hardware parallelism (e.g., 32 threads on a 16-thread system) may lead to oversubscription effects such as higher context switching and cache contention, explaining why scaling does not monotonically improve at high T .

6.2. Conclusions

This project delivered a complete sequential baseline and two OpenMP parallel implementations of K-Means, enabling a controlled comparison between AoS and SoA memory layouts under the same algorithmic structure. Correctness was validated using a dedicated test harness, showing consistent SSE values and identical convergence iterations between sequential and parallel executions. On the performance side, the best configuration found is the OpenMP–SoA variant

with `static` scheduling and `chunk=0`, which achieves a peak speedup of approximately $4.6\times$ on the largest dataset ($N = 10^6$, $K = 20$), with the most favorable scaling around 8 threads.

The main limitations of the current study are the focus on a single shared-memory machine and on synthetic 2D datasets, ...as well as the lack of a full end-to-end evaluation including dataset loading/parsing and centroid initialization, which were excluded from the timed region. Future work could extend the evaluation to higher-dimensional data, larger and more diverse datasets, and multi-socket or NUMA systems; moreover, additional optimizations could be explored, such as reducing the cost of the centroid merge step, improving SIMD vectorization of the distance loop, using more efficient memory allocators for thread-local buffers, and investigating alternative parallelization strategies (e.g., parallel reductions over clusters or blocking techniques to improve cache reuse).

References

- [1] J. MacQueen, *Some Methods for Classification and Analysis of Multivariate Observations*, Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, 1967.
- [2] S. Lloyd, *Least Squares Quantization in PCM*, IEEE Transactions on Information Theory, 28(2), 1982.
- [3] J. A. Hartigan and M. A. Wong, *Algorithm AS 136: A K-Means Clustering Algorithm*, Journal of the Royal Statistical Society. Series C (Applied Statistics), 28(1), 1979.
- [4] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities*, AFIPS Conference Proceedings, 1967.
- [5] J. L. Gustafson, *Reevaluating Amdahl’s Law*, Communications of the ACM, 31(5), 1988.
- [6] OpenMP Architecture Review Board, *OpenMP Application Programming*

Interface Specification. (Online)

<https://www.openmp.org/specifications/>

- [7] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2011.